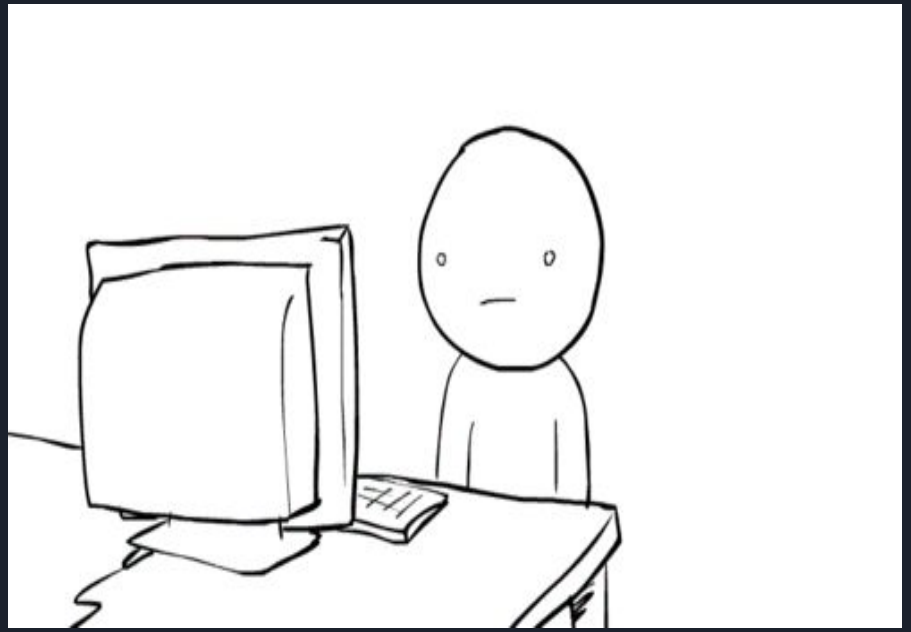
Summarizer
14/02/2023

Tiago  Marques

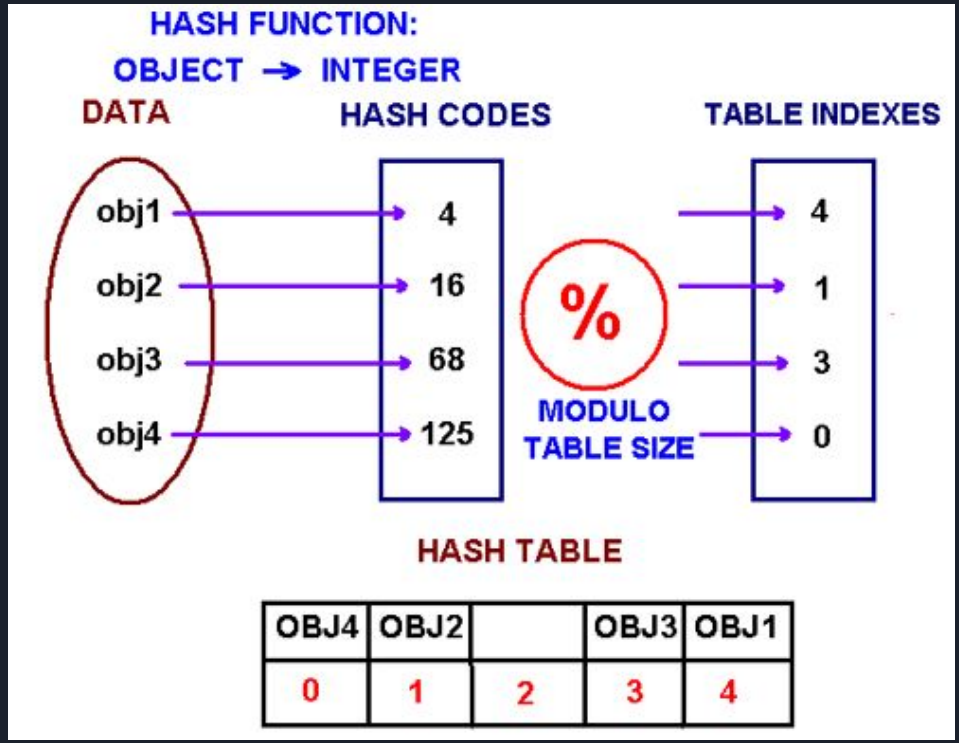
Table of Contents

What is Hash???

My fist research on the web this is the result.



Hashing is defined as the process of transforming one value into another based on a particular key. A *hash* is a function that converts an input value into an output value that is usually shorter, and is designed to be unique for each input value. Although collisions are unavoidable, your hash function should attempt to reduce collisions, which implies that different input values should not generate the same hash code.



Map Interfaces

- HashMap
- TreeMap
- LinkedHashMap

```java
HashMap<String, String> capitalCities = new HashMap<String,String>();

        // Add keys and values (Country, City)
        capitalCities.put("England", "London");
        capitalCities.put("Germany", "Berlin");
        capitalCities.put("Norway", "Oslo");
        capitalCities.put("USA", "Washington DC");
        System.out.println(capitalCities);
```

```
"C:\Users\Vila Carminho\.jdks\corretto-1.8.0_362\bin\java.exe" ...
{USA=Washington DC, Norway=Oslo, England=London, Germany=Berlin}

Process finished with exit code 0
```

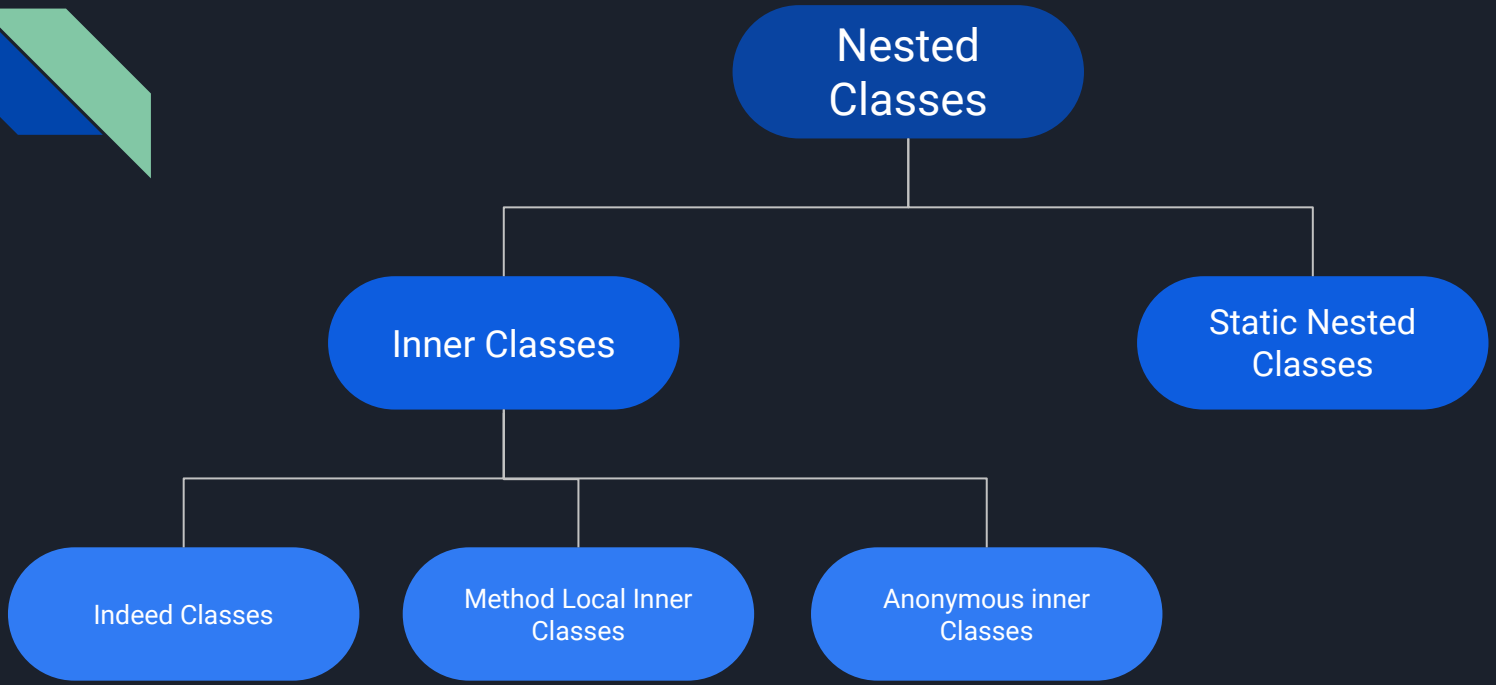
```java
HashMap<String, String> capitalCities = new LinkedHashMap<String, String>();

// Add keys and values (Country, City)
capitalCities.put("England", "London");
capitalCities.put("Germany", "Berlin");
capitalCities.put("Norway", "Oslo");
capitalCities.put("USA", "Washington DC");
System.out.println(capitalCities);
```

```
"C:\Users\Vila Carminho\.jdks\corretto-1.8.0_362\bin\java.exe" ...
{England=London, Germany=Berlin, Norway=Oslo, USA=Washington DC}

Process finished with exit code 0
```

Nested Class


Nested Classes
Inner Classes
Static Nested Classes
Indeed Classes
Method Local Inner Classes
Anonymous inner Classes

```
class OuterClass{
  ...
  public static class StaticNestedClass{
  ...
  }
}
```

```
class OuterClass{
  ...
  public void method(){
    class LocalClass{
      ...
    }
  }
}
```

```
class OuterClass{
  ...
  class InnerClass{
     ...
  }
}
```

```
class OuterClass{
  ...
  public void method(){
    ClassType anonymousClass = new ClassType(){
      ...
    }
  }
}
```

Class Exercise implementation

Problem use the range exercise to print the iterator in the reverse order..

```
public void setReversed(boolean reversed) {
    this.reversed = reversed;
}
```

```
    @Override
public Iterator<Integer> iterator() {
    if (!reversed) {
        current = min - 1;
    } else current = max + 1;
    return new Iterator<Integer>()
```

```java
 @Override
public boolean hasNext() {
    if (!reversed) {
// Contagem Crescente
    if (current < max) {
       return true;
      }
       return false;
                 }
//Comtagem Decrescente
     if (current > min) {
       return true;
      }
       return false;
  }
```

```java
@Override
public Integer next() {
    if (!reversed) {
          current++;
          return current;
       }
          current--;
          return current;
}
```

```java
import java.util.Iterator;

/**
 * Created by ferrao on 30/05/16.
 */
public class RangeTest {

    public static void main(String[] args) {

        Range r = new Range(1, 10);
        r.setReversed(true);

        System.out.println("--- USING ITERATOR ---");


        for (Integer i : r) {
            System.out.println("Iterated: " + i);
        }

    }

}
```

RangeTest

```
"C:\Users\Vila Carminho\.jdks\corretto-1.8.0_362\bin\java.exe" ...
--- USING ITERATOR ---
Iterated: 10
Iterated: 9
Iterated: 8
Iterated: 7
Iterated: 6
Iterated: 5
Iterated: 4
Iterated: 3
Iterated: 2
Iterated: 1

Process finished with exit code 0
```

```java
import java.util.Iterator;

/**
 * Created by ferrao on 30/05/16.
 */
public class RangeTest {

    public static void main(String[] args) {

        Range r = new Range(1, 10);
        r.setReversed(false);

        System.out.println("--- USING ITERATOR ---");


        for (Integer i : r) {
            System.out.println("Iterated: " + i);
        }

    }

}
```

RangeTest

```
--- USING ITERATOR ---
Iterated: 1
Iterated: 2
Iterated: 3
Iterated: 4
Iterated: 5
Iterated: 6
Iterated: 7
Iterated: 8
Iterated: 9
Iterated: 10

Process finished with exit code 0
```

Thank you !!!